

Nombre y Apellido: _____ Nro. CIC: _____

Tema 1 (10p)

Considere un grafo $G = (V, E)$ no dirigido. Implemente un algoritmo de tiempo $O(|V| + |E|)$ en el peor caso, que retorna TRUE si el grafo esta conectado y FALSE en caso contrario. Justifique el tiempo solicitado.

Considere que ya disponibles las operaciones básicas sobre un grafo (adyacentes, cantidad de vértices, lista de vértices, etc).

Tema 2 (10p)

El camino más largo en un GDA (Grafo dirigido acíclico) es un camino dirigido con la máxima cantidad de vértices. Describa un algoritmo, puede ser en pseudocódigo, que encuentre el camino más largo en un GDA. Considerando el GDA como $G=(V,E)$ su algoritmo debe encontrarse en $O(|V| + |E|)$. Se debe retornar la secuencia de vértices en el orden del camino encontrado. Si existen más de un camino entonces retorne cualquiera. Justifique el tiempo de su algoritmo.

Considere que ya disponibles las operaciones básicas sobre un grafo (adyacentes, cantidad de vértices, lista de vértices, etc).

Tema 3 (20p)

Indique con Falso (F) o Verdadero (V) las expresiones que se listan a continuación. Fundamente ambas respuestas.

- a) ____ Existe un algoritmo de ordenación basado en comparación que usa 8 comparaciones en el peor caso para ordenar 5 elementos..
- b) ____ En una estrategia de diseño de algoritmo utilizando programación dinámica es posible utilizar un abordaje top-down para la solución de los subproblemas.
- c) ____ Cuando es aplicado el algoritmo de *MergeSort* a un arreglo de N elementos distintos, el número de llamadas a *combinar()* es alrededor de N.
- d) ____ El número de nivel generado aleatoriamente en un SkipList sigue una distribución geométrica.
- e) ____ Una solución óptima al problema de la mochila siempre contendrá el objeto i con el mayor ratio de $\text{peso}_i/\text{beneficio}_i$.
- f) ____ El valor esperado del número de cruces obtenido al lanzar 10 (veces) veces una moneda es 5.
- g) ____ Utilizando una tabla de dispersión, podemos crear un algoritmo de ordenación similar a *CountingSort* que ordena un conjunto de elementos de enteros (no restringidos) en tiempo lineal. El algoritmo funciona igual que *CountingSort*, excepto que usa un hash de cada entero como índice dentro de la tabla utilizada por *CountingSort*.
- h) ____ El algoritmo *RadixSort* NO funciona correctamente, es decir no ordena, si nosotros ordenamos cada dígito individual usando el algoritmo *InsertSort* en vez del algoritmo *CountingSort*.
- i) ____ La codificación binaria $\{0, 01, 11, 1\}$ que corresponde a un alfabeto de 4 elementos es aceptable.
- j) ____ Suponga que desea almacenar datos en un B+Arbol teniendo en cuenta 4096 bytes de tamaño de bloque de disco, 96 bytes de clave y 4 bytes por puntero. Entonces el valor óptimo para el orden del árbol es 41..

Tema 4 (5p)

Asuma que un entero de n bits (representado por la notación binaria estándar) toma un valor entre 0 y $2^n - 1$ con igual probabilidad:

- a) Por cada posición, ¿Cuál es la probabilidad de que el valor del bit sea 1 y cuál es la probabilidad de que sea 0? (1p)
- b) ¿Cuál es el número esperado de bits "1" para un número aleatorio de n bits? (2p)
- c) ¿Cuál es el valor esperado para la posición del bit "1" más a la izquierda? En otras palabras, cuántas posiciones en promedio debemos examinar de izquierda a derecha antes de encontrar un bit "1". (2p)

Tema 5 (10p)

Considere el algoritmo de Kruskal que utiliza la estructura *Disjoint-Set*.

```
MST-KRUSKAL( $G, w$ ) // Recibe el Grafo  $G$  y los pesos  $w$  de cada arista
1   $A \leftarrow \emptyset$ 
2  Para cada vertice  $v \in V[G]$  hacer
3      MAKE-SET( $v$ )
4      ordenar crecientemente los vertices de  $E$  por el peso  $w$ 
5      Para cada arista  $(u, v) \in E$ , tomado en orden creciente por peso hacer
6          Si FIND-SET( $u$ ) = FIND-SET( $v$ ) entonces
7               $A \leftarrow A \cup \{(u, v)\}$ 
8              UNION( $u, v$ )
9          fin-Si
10     fin-Para
11 fin-Para
12 retorna  $A$ 
```

Diseñe, como una clase Java, la estructura de Datos para Conjuntos disjuntos (*Disjoint-Set*) (2p) y los algoritmos de MAKE-SET (2p), FIND (2p) y UNION (2p). Indique, con su diseño de estructura de datos, cuál es el tiempo en términos de O del algoritmo de Kruskal (2p).

Tema 6 (5p)

Considerar el algoritmo de ordenación por Inserción (escrito en Java) como el que se muestra a continuación:

```
void InsertSort( int [ ] A ) {
    int n = A.length;
    for ( int i = 1; i < n ; i++ ) {
        int v = A[i];
        int j = i-1;
        while ( j >= 0 && A[j] > v ) {
            A[j+1] = A[j];
            j--;
        }
        A[j+1] = v;
    }
}
```

Calcule el valor esperado del número de comparaciones. Hágalo de manera formal.

Tema 7 (5p)

Considere la matriz $A = \begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix}$. Es posible calcular el i -ésimo término de la serie fibonacci F_i , dado $i \geq 1$,

$A^i = \begin{pmatrix} F_{i-1} & F_i \\ F_i & F_{i+1} \end{pmatrix}$. Considere el hecho de que $A^{2i} = (A^i)^2$ y que $A^{2i+1} = A \cdot (A^i)^2$. De acuerdo a esto, diseñe

un algoritmo divide y vencerás para calcular el i -ésimo término de la serie fibonacci F_i . Escriba la ecuación de recurrencia e indique el tiempo asintótico de un solución.